Report

Team Name: Krishiv
Team Members and Roll no.: Krishiv Agrawal - EE25B074

1 Summary

I first pre-processed and cleaned the data, then aggregated shift levels features and then experimented with different models (XGBoost, Catboost, etc.) to predict fuel consumption. I finally used catboost with some post processing (squashing impractical predictions).

This was my first hackathon with real world messy data. All the hackathons i tried before this had synthetically generated data where training the model was pretty easy, so I did get to learn a lot from this about data cleaning, feature engineering and model training.

2 Problem Understanding

The primary target was to predict the fuel consumption of a dumper on a shift level basis. The target variable is the fuel consumed by a specific dumper on a certain date in some shift.

The challenging part was that the real world data was given which had irregularities in some places. Finding and cleaning those was very important for the model to learn the underlying pattern correctly.

Also, the fuel consumed depends on many different factors; the terrain and the route, acceleration and braking patterns of the operator, the altitude gained, the stop-go intensity etc. Finding which features were important for the model was again challenging and interesting.

Determining dump events from analog signals, spatial analysis and mapping the routes taken by dumpers was also very challenging.

3 Dataset Overview

- **Brief overview :**
  1. 32 different dumpers, and total of 23277877 timestamps.
  2. Jan 1-20, Feb 1-20 and March 1-10 was used for training and validation, while Jan 21-31, Feb 21-28, March 11-20 was used for testing.
  3. Around 20% rows with 0 fuel consumption.
- **Key Features Used:** I broadly used features in 3 categories
  1. Hours based : ignition, moving, idle, uphill, downhill, flat hours and the 'ratio' of moving and ignition hours (useful for idle shifts).
  2. Distance based : total distance, cumulative altitude gain and loss

3. Speed and gradients : sum of speed values in a shift, sum of accn_x values, stop-go intensity and gradient (altitude gain/dist).

- **Notable Observations:**

1. 36% of engine hours were just idling hours, the truck moved only in the rest 64% time.
2. Surprisingly, the net altitude change isn't a very useful feature since dumpers generally go up and down a mine and end up at about the same altitude. Using cumulative altitude gain and cumulative altitude loss is a much better feature.
3. So number of altitude cycles can be used as a proxy for total trips.
4. The ignition sensor of Dump033 was broken in the test dataset, for which i had to manually change its ignition state.

4 Methodology

4.1 Data Preprocessing

I tried out different filters and found the best ones to be

1. Number of satellites > 7
2. Some trucks had a broken ignition sensor (Dump033), so I set ignition $= 1$ if speed > 2 for such cases.
3. Dropped rows with no lat long values and created a 'working_date' column by adding 2 hours to the given timestamps.
4. Gnss pdop signal < 2.5
5. I tried restricting gnss hdop too and wasted a lot of time wondering where my data went, it was because january data didn't have gnss hdop values.

After aggregating features on a shift level, some shifts had features that weren't physically possible, so i cleaned those :

1. Some rows had engine on for over half an hour, yet showed 0 fuel consumption, i deleted those.
2. Some had engine on for only 5-10 minutes, yet showed significant fuel consumption.
3. Rows with cumulative altitude gain < 500m but over 100L of fuel.
4. I defined a temporary column fuel_efficieny, and deleted all rows with efficiency > 7.5.
5. I also trained my model only on rows with fuel consumption > 2. Since 20% of training set had 0 fuel consumption, it would skew the model predictions.

4.2 Feature Engineering

Derived broadly 4 new types of features, each calculated for a shift.

1. **Categorical :**

- Vehicle
- Shift

2. **Hours based :**

   - ignition_hrs : hours where ignition = 1
   - moving_hrs : ignition = 1 and speed > 1.
   - idle_hrs : ignition = 1 and speed < 1.
   - uphill_hrs : altitude difference > 0.5 and speed > 1.
   - downhill_hrs : altitude difference < -0.5 and speed > 1.
   - flat_hrs : altitude difference between 0.5 and -0.5 with speed > 1.
   - ratio : moving_hrs / ignition_hrs. (useful feature in idle shifts).

3. **Distance based :**

   - total distance : sum of total haversine distance in a shift.
   - cumulative altitude gain : sum of all altitude differences > 0 .
   - I tried using standard deviation of altitude as a feature but it didn't lead to significant improvement.

4. **Speed and gradients :**

   - Sum of speed values in a shift.
   - Sum of gradients (altitude change/distance) in a shift.
   - The stop go intensity : occurences when the speed of a truck was > 1 and became < 1 in the next timestamp.
   - ax_sum : Sum of acceleration in x direction

5. **Some interesting ones :**

   - Average historical mileage (fuel efficiency) of a truck.
   - I found average fuel consumed by each truck isn't a good feature since it leads to over prediction in idle shifts.
   - ratio * total distance (can be used as a proxy of total weight)

6. I found values summed over a shift perform better than their mean over the shift. Using the sum of speed values is better than using mean speed as a feature.

4.3 Modelling Approach

I tried out XGBoost, random forest and catboost. I finally used catboost since there were categorical features in the dataset which catboost natively handles and performs better on. It also gave the lowest MSE loss (around 300 on val set).

I had a 2 phase approach :

1. All shifts with ignition_hrs < 0.1 and total distance < 0.1 were post processed to 0L of fuel. I did this because the model usually predicted around 30L fuel for these cases, which is obviously false.
2. For all the other shifts, I directly used the model prediction.

   The Catboost model achieved an rmse of around 20.

4.4 Validation Strategy

I used a time based split instead of a random split. In a random split, the model can see shifts from Monday and Wednesday during training, making it predict Tuesday's consumption for the same truck, leading to a low validation MSE that does not reflect real world performance. A temporal split forces the model to train on earlier dates (Jan 1–16) and predict on future dates (Jan 17–20). While this increases the validation loss compared to a random split, it provides a better estimate of generalized performance.

5 Secondary Outputs

I tried making a classifier to predict dump events, plot routes based on the given maps, but didn't get any significant improvements in model prediction.

1. **Route-Level Fuel Benchmark:** I didn't use a aggregated route level feature, but different terrain features. Although it could easily be aggregated as follows.

```
    # Each shift has a terrain fingerprint
route_features = ['total_dist', 'cum_altitude_gain', 'stop_go', '
    uphill_hrs']

# Cluster into 10 route types
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=10)
df['route_type'] = kmeans.fit_predict(df[route_features])

# Average fuel per route type, independent of truck
route_fuel = df.groupby('route_type')['acons'].mean()
df['route_expected_fuel'] = df['route_type'].map(route_fuel)
```

2. **Dumper Efficiency Component:** Adding vehicle as a categorical feature captured some of the dependency on vehicle, but the maximum improvement came from adding historical average mileage of each truck as a feature. I found average fuel consumed by each truck isn't a good feature since it leads to over prediction in idle shifts.
3. **Cycle Segmentation Methodology:** I tried training binary classifier that took around 20 timestamps as input and predicted if a dump event occurred in it. It had huge errors so I discarded it. It was because analog signals were present only for trucks in mine001 so location couldn't be a good way to predict dump events, hence the classifier relied on ignition_hrs and distance as its primary features. But it was easy to trick since getting stuck in traffic showed the same features as dumping (ignition on and speed 0).
4. **Daily Fuel Consistency:** My model gave an rmse loss of around 17 on validation set.

6 Key Findings & Insights

Key insights :

- Analog signals were given only for trucks in mine001, so finding dump locations and predicting dump events using that wasn't an option since no dump locations for mine002 were known.
- Surprisingly, the net altitude change isn't a very useful feature since dumpers generally go up and down a mine and end up at about the same altitude. Using cumulative altitude gain and cumulative altitude loss is a much better feature. The total trips can even be estimated from up and down altitude cycles.
- A significant number of shifts were idle shifts, where the moving_hrs is pretty low but ignition_hrs is high. This could be due to maintainence, repair or traffic.
- Training the model on shifts with fuel consumption > 2 improves it performance compared to training it on all of the dataset. This is because around 20% of the dataset has fuel consumption $= 0$, which skews the model predictions.
- So around 500 rows in the training dataset were removed because of cleaning filters.
- The most important features were uphill_hrs, total distance and cumulative altitude gain. I noticed how trucks lose altitude going to mine and gain altitude while climbing up, which is a very good proxy for how many cycles they did.

Personal learnings :

- Robust feature engineering and data cleaning is super important.
- To go through model predictions and checking if they are practically possible.
- Cleaning training data, adding historical mileage as a feature and squashing impractical outputs to 0 improved model performance a lot.

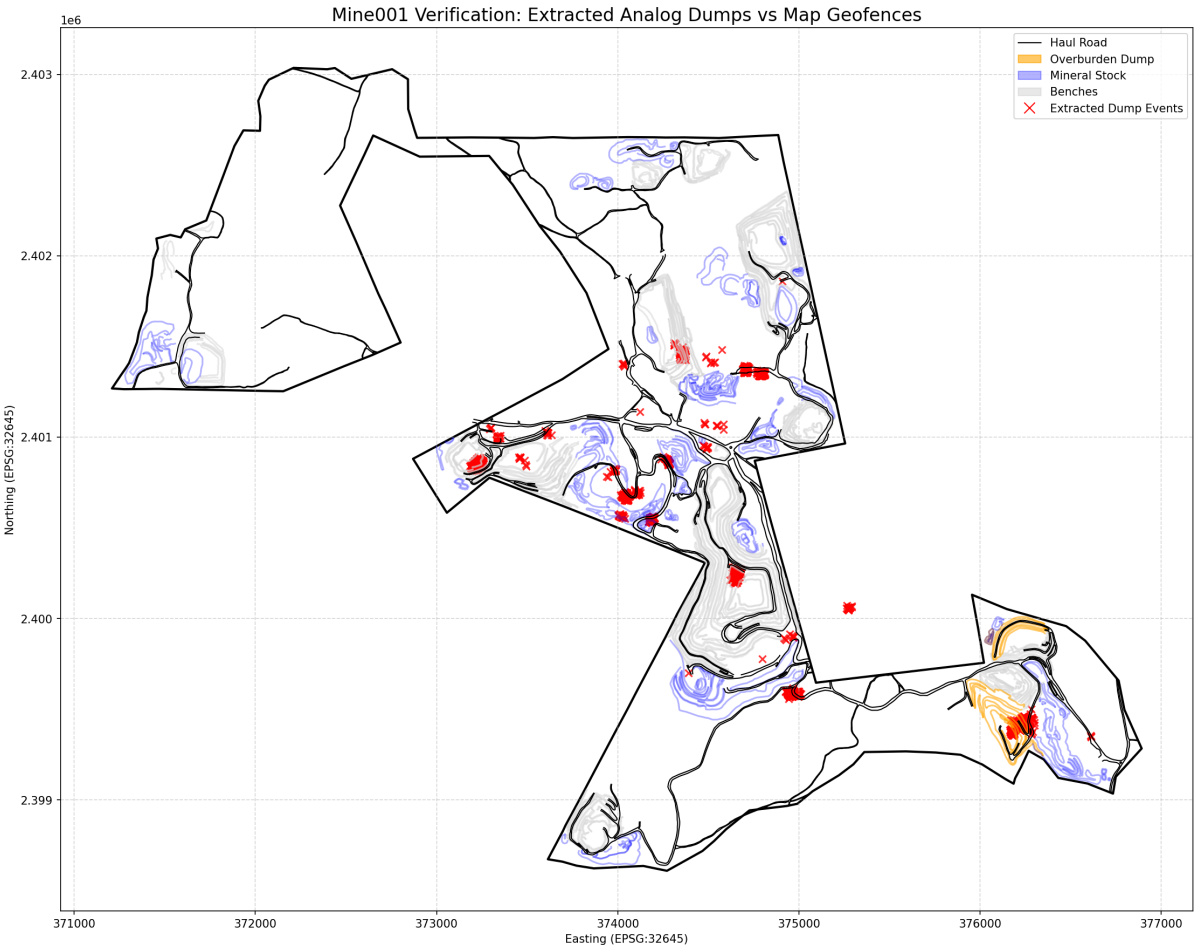
7 Codebase

- **Repository Link:** `https://github.com/KrishivAgrawal1/AI_Guild_2026`
- **Code Organisation:**

```
root/
├── data/
├── src/
│   └── code.ipynb
└── report.pdf
```

- **Reproducibility:**

  Environment : Python 3.11.14, pandas 2.3.3, numpy 2.3.4, scikit-learn 1.8.0, catboost 1.2.10, xgboost 3.2.0

  Steps :
  Place all data files in ./data/
  Run the jupyter notebook.

- **Discarded Approaches:** I inferred the dump events from march data and plotted them on mine001, red crosses represent dumping points. Some dumping locations were outside the mine boundary, hence I didn't further use it.